FixPath

A Decision-First Troubleshooting Platform for Everyday Problems

Haoyu Xiong (817032)

1. Design Goal

FixPath is a web-based decision-support platform that helps users report everyday device or household problems, receive structured troubleshooting guidance, and escalate to experts when the situation is unsafe, uncertain, or beyond the scope of self-service. The design goals below describe the non-functional requirements that shaped the system and explain how each was addressed.

1.1 Usability

The platform must be straightforward enough for non-technical users to submit a problem report, interpret a diagnosis result, and choose an appropriate next step without confusion.

Implementation:

- A step-by-step submission form collects device category, manufacturer, model, problem description, and observed symptoms.

- A keyword-based symptom suggestion feature analyzes the user's problem description and returns a ranked list of relevant symptoms, reducing the effort required to identify and select the correct ones.

- Diagnosis results are presented in plain language, including the detected risk level, recommended action, outcome type, and a structured explanation of the decision.

- Clear action options — such as self-troubleshooting guidance, expert consultation, or replacement recommendation — are displayed after each diagnosis so that users can proceed to the appropriate next stage.

- The interface layout remains consistent across the submission, diagnosis, consultation, and replacement recommendation views.

1.2 Performance

The system must generate a diagnosis result quickly after the user submits a problem report.

Implementation:

- The backend uses a lightweight rule-based scoring algorithm rather than computationally intensive machine learning models, keeping diagnosis response time low.

- Only decision rules associated with the reported device are loaded and evaluated, avoiding unnecessary database queries.

- Structured report, rule, and result data are stored in a relational MySQL database to support efficient retrieval.

- Frontend and backend responsibilities are separated: the React frontend remains responsive while the Express backend processes the diagnosis independently.

1.3 Safety

The platform must reduce the risk of unsafe self-repair attempts and avoid providing misleading guidance in dangerous situations.

Implementation:

- The Diagnosis Engine includes safety override logic that runs before standard rule evaluation. If a high-risk symptom such as smoke or burning smell is detected, or if two or more high-severity symptoms are reported simultaneously, the system immediately assigns an emergency stop outcome and stops further self-troubleshooting guidance.

- Cases where the diagnosis is ambiguous — defined as two candidate rules whose scores differ by fewer than eight points — are escalated to expert consultation rather than producing an uncertain automated recommendation.

- Decision rules include a RequiresExpert flag. When this flag is set, the engine forces an expert consultation outcome regardless of the symptom score.

- The Diagnosis Engine is scoped to structured decision support and does not attempt

to replicate professional technical judgment.

1.4 Security

The platform must protect user accounts, submitted reports, and consultation-related information from unauthorized access or data exposure.

Implementation:

- User and expert passwords are stored as bcrypt-hashed values. Plain-text passwords are never written to the database.

- All database queries use parameterized statements via the mysql2 driver, preventing SQL injection.

- Backend input validation checks that required fields are present and that field values are within expected types and formats before any database operation is performed.

- File uploads are validated against a whitelist of permitted MIME types (JPEG, PNG, WebP, MP4, QuickTime, WebM) and are subject to a per-file size limit of 20 MB, preventing unsupported or excessively large files from being stored.

- Database credentials are stored in environment variables using dotenv and are not included in the source code.

- All data operations pass through backend API endpoints; the database is not directly accessible from the frontend.

1.5 Scalability and Maintainability

The system should support future expansion in the number of users, devices, troubleshooting rules, experts, and replacement options without requiring structural redesign.

Implementation:

- The system is organized into distinct layers: React frontend, Express backend, and MySQL database, with each layer communicating through defined RESTful API contracts.

- The Diagnosis Engine is modular: new decision rules and device categories can be inserted into the database without modifying the engine's evaluation logic.

- Rules, reports, diagnosis results, consultations, and replacement options are stored in separate relational tables, reducing coupling and improving maintainability.

- The backend uses a connection pool (limit of ten connections) to manage database resources efficiently under concurrent load.

1.6 Clear System Boundaries

FixPath must have clearly defined responsibility boundaries so that its scope remains realistic and technically achievable within this project.

Implementation:

- FixPath provides structured decision support, expert referral, and merchant redirection.

- The platform does not perform physical repair work, does not process product purchases, and does not guarantee service outcomes.

- Expert consultation and merchant transactions occur outside the platform. FixPath provides the user with the appropriate contact or link after a diagnosis decision is made.

2. System Architecture

FixPath uses a three-layer web architecture consisting of a React frontend presentation layer, a Node.js and Express backend application layer, and a MySQL relational database layer. This architecture supports the core platform workflow: problem submission, rule-based diagnosis, expert escalation, and replacement recommendation.

2.1 Frontend Presentation Layer

The frontend is implemented as a React web application. It is responsible for collecting user input and presenting processed results in a clear and structured format. The frontend supports the following functions:

- user and expert authentication,
- problem report submission with media upload,
- keyword-based symptom suggestion display,
- diagnosis result presentation,
- expert consultation requests and status tracking,
- replacement recommendation display,
- user and expert profile management,
- feedback submission.

The frontend communicates with the backend through HTTP API calls using Axios, and renders the returned diagnosis, consultation, or replacement data dynamically.

2.2 Backend Application Layer

The backend is implemented using Node.js and Express.js. It validates input, executes business logic, interacts with the database, and returns structured JSON responses to the frontend. The backend is organized into the following major modules:

Diagnosis Engine

The Diagnosis Engine is the core decision module of FixPath. When triggered, it loads the problem report and its associated symptoms from the database, retrieves all active decision rules for the reported device, and evaluates each rule using a weighted scoring algorithm.

The scoring formula incorporates:

- symptom severity weights (Low: 1 point, Medium: 2 points, High: 4 points),
- a bonus for exact symptom set matches,
- a priority bonus based on the rule's configured priority,
- a penalty for extra symptoms reported beyond those required by the rule.

Safety overrides are evaluated before the scoring phase. If a critical symptom (smoke or

burning smell) is present, or if two or more high-severity symptoms are reported, the engine immediately returns an emergency stop outcome without proceeding to rule scoring. If the top two candidate rules score within eight points of each other, the result is classified as ambiguous and escalated to expert consultation.

The engine stores the final outcome, recommendation, risk level, confidence score, and explanation in the DECISION_RESULT table.

Risk Detection

Risk detection is integrated directly into the Diagnosis Engine rather than implemented as a separate service. High-risk indicators and ambiguity conditions are evaluated at the start of each analysis run, ensuring that unsafe cases are never routed to self-troubleshooting guidance.

Expert Escalation Module

This module handles cases that require professional assistance. It creates a CONSULTATION record linked to the diagnosis result and the requesting user, and automatically assigns an available expert by querying the EXPERT table for experts with AvailabilityStatus = 'Available'. Consultation status transitions (pending, ongoing, completed) are managed through a dedicated status update endpoint.

Merchant Referral Module

When the diagnosis result suggests replacement, this module retrieves replacement options from the REPLACEMENT_OPTION table that are associated with the matched rule and device. Each option includes the product name, estimated price, and a direct link to the merchant's listing.

Symptom Suggestion Module

This module accepts a device identifier and the user's problem description, and returns a ranked list of symptoms relevant to that device. The ranking is calculated using keyword scoring: the problem description is normalized and compared against a predefined

keyword map for each symptom. Matches on full keyword phrases, individual words, symptom names, and severity level contribute to each symptom's score. The result is a list of symptoms ordered by relevance, which the frontend presents as suggestions during report submission.

RESTful API Endpoints

The backend exposes the following primary API routes:

Method	Endpoint	Description
POST	/auth/user-login	Authenticate a user
POST	/auth/expert-login	Authenticate an expert
POST	/report	Submit a problem report with optional media upload
POST	/diagnose/:reportId	Trigger diagnosis for a submitted report
POST	/symptoms/suggest	Return keyword-scored symptom suggestions
GET	/devices/categories	Retrieve all device categories
GET	/devices/category/:categoryId	Retrieve devices in a category
POST	/consultation	Create a consultation and assign an expert
PATCH	/consultation/:id/status	Update consultation status
GET	/consultation/expert/:expertId	Get consultations assigned to an expert
GET	/replacement/:resultId	Get replacement options for a diagnosis result
POST	/feedback	Submit feedback for a diagnosis or consultation

2.3 Database Layer

The database layer is implemented using MySQL and stores all structured data required by the FixPath workflow, including:

- user and expert accounts,
- device categories and devices,
- symptoms,
- problem reports and associated media,
- problem report-symptom records,

- decision rules and rule-symptom mappings,
- diagnosis results,
- consultation records,
- merchant records and replacement options,
- user feedback.

The database supports persistent storage, rule retrieval during diagnosis, result recording, consultation tracking, and replacement recommendation management.

2.4 Main System Request Flow

The primary FixPath workflow operates as follows:

1. The user selects a device category and device, enters a problem description, and optionally receives keyword-based symptom suggestions from the backend.
2. The user selects observed symptoms, optionally uploads supporting media, and submits the problem report.
3. The backend validates the request and writes the report, symptom associations, and media references to the database atomically using a transaction.
4. The frontend sends a diagnosis request for the created report.
5. The Diagnosis Engine checks for safety override conditions, then loads and scores applicable decision rules.
6. A DECISION_RESULT record is created containing the outcome, risk level, recommendation, confidence score, and explanation.
7. The backend returns the result to the frontend.
8. The frontend displays the result and presents the appropriate next step: self-troubleshooting guidance, expert consultation, or replacement recommendation.

2.5 Rule-Based Decision Logic

The Diagnosis Engine classifies each case into one of the following outcome categories:

- **low-risk troubleshooting** — the user is guided through self-repair steps,

- **stop and inspect** — the user is advised to stop using the device and allow it to cool or stabilize,

- **replace part** — the diagnosis suggests a component replacement,

- **expert consultation** — the case is escalated to an available expert,

- **emergency stop** — use of the device must cease immediately due to high-risk indicators.

Each decision rule is stored in the DECISION_RULE table and is associated with one or more symptoms through the RULE_SYMPTOM junction table. During evaluation, the engine loads active rules for the reported device, scores each rule based on symptom overlap and severity weighting, applies safety overrides where applicable, and selects the highest-scoring rule as the diagnosis outcome.

Example rules:

- If device = Vacuum Cleaner and symptom = Weak suction → outcome = low-risk troubleshooting

- If device = Heater and symptom = Smoke → outcome = emergency stop (via safety override)

- If reported symptoms are ambiguous across the top candidate rules → outcome = expert consultation

This approach keeps the Diagnosis Engine within a clearly defined and controllable scope, while fulfilling the platform's goal of structured decision assistance.

3. Hardware and Software Configuration

3.1 Development Environment

Hardware:

- Manufacturer: Acer

- Processor: 11th Gen Intel Core i7-11700F@ 2.50 GHz
- Installed RAM: 16.0 GB
- System Type: 64-bit, x64-based processor

Operating System:

- Windows 11 Home, Version 25H2

Development Software:

- Visual Studio Code — primary development environment
- React.js, HTML, CSS, and JavaScript — frontend development
- Node.js and Express.js — backend development
- MySQL — relational database management
- Git and GitHub — version control
- Postman — API testing and validation
- Balsamiq and draw.io — interface and diagram design

This environment is sufficient for building and testing the FixPath prototype. The system's primary operations — form submission, rule evaluation, database interaction, and result presentation — do not require high-performance computation.

3.2 Proposed Deployment Environment

FixPath is designed for cloud-based prototype or small-scale deployment.

- **Platform:** PaaS environment (Render)
- **Frontend Hosting:** Static hosting for the React production build
- **Backend Runtime:** Node.js server instance
- **Database:** Managed MySQL database
- **Minimum Server Resources:** 2 vCPU and 2 GB RAM for prototype deployment
- **Client Access:** Desktop or mobile web browser

3.3 Security Configuration

The following security measures are implemented in the current prototype:

- Passwords are stored as bcrypt hashes; plain-text passwords are never persisted
- All SQL queries use parameterized statements to prevent injection
- Backend input validation checks required fields and value types before database operations
- File upload validation enforces an allowed MIME type whitelist and a 20 MB per-file limit
- Database credentials are stored in environment variables and excluded from source control
- HTTPS is recommended for production deployment to secure frontend-to-backend communication

3.4 Suitability of the Proposed Configuration

This configuration is appropriate for FixPath because the system's core operations — structured form submission, keyword-based symptom suggestion, rule scoring, result generation, consultation management, and merchant redirection — do not require heavy computation. The main requirements are reliable database access, modular RESTful APIs, and secure credential handling, all of which the proposed configuration satisfies at prototype scale.

4. Database Design

The FixPath database is a relational schema that supports the complete troubleshooting workflow, from problem submission through diagnosis, escalation, replacement recommendation, and feedback collection. Entities are separated to reduce redundancy and improve maintainability.

4.1 Entity Structure

USER(

UserID PK,

Name,

Email UNIQUE,

PasswordHash,

CreatedTime

)

DEVICE_CATEGORY(

CategoryID PK,

CategoryName,

CategoryDescription

)

DEVICE(

DeviceID PK,

CategoryID FK,

DeviceName,

Manufacturer,

Model,

DeviceDescription

)

SYMPTOM(

SymptomID PK,

SymptomName,

SymptomDescription,

SeverityLevel

)

PROBLEM_REPORT(

ReportID PK,

UserID FK,

DeviceID FK,

ReportDate,

ProblemDescription,

DetectedRiskLevel,

Status,

MediaURL

)

PROBLEM_REPORT_SYMPTOM(

ReportID FK,

SymptomID FK,

Notes,

PRIMARY KEY (ReportID, SymptomID)

)

REPORT_MEDIA(

MediaID PK,

ReportID FK,

MediaType,

MediaURL,

UploadedTime

)

DECISION_RULE(

RuleID PK,

DeviceID FK,

OutcomeType,

GuidanceText,

RiskLevel,

Priority,

RequiresExpert,

IsActive

)

RULE_SYMPTOM(

RuleID FK,

SymptomID FK,

PRIMARY KEY (RuleID, SymptomID)

)

DECISION_RESULT(

ResultID PK,

ReportID FK,

RuleID FK,

Outcome,

Recommendation,

Explanation,

Confidence,

ResultDate,

RiskLevel,

RequiresExpert

)

EXPERT(

ExpertID PK,

ExpertName,

ServiceType,

Phone,

ExpertEmail,

PasswordHash,

ExpertRating,

AvailabilityStatus

)

CONSULTATION(

ConsultationID PK,

ResultID FK,

UserID FK,

ExpertID FK,

RequestType,

ConsultationStatus,

RequestDate,

ConsultationDate,

MeetingLink,

ConsultationNotes

)

MERCHANT(

MerchantID PK,

MerchantName,

Website,

ContactEmail

)

REPLACEMENT_OPTION(

OptionID PK,

RuleID FK,

DeviceID FK,

MerchantID FK,

ProductName,

ProductType,

EstimatedPrice,

ProductURL

)

FEEDBACK(

FeedbackID PK,

ReportID FK,

UserID FK,

ResultID FK,

ConsultationID FK,

ExpertID FK,

FeedbackType,

Comments,

Rating,

SubmittedDate

)

4.2 Database Design Explanation

Users, Devices, and Problem Reports

A USER submits a PROBLEM_REPORT for a selected DEVICE. Each device belongs to one DEVICE_CATEGORY, allowing the system to organize devices by type such as vacuum cleaners, heaters, or sinks.

A problem report stores the reporting user, the affected device, the problem description, the detected risk level, the current status, and references to any uploaded media. This provides full traceability from user input through to diagnosis result.

Symptoms and Symptom Mapping

A problem report may include multiple observed symptoms, and the same symptom may appear across many reports. PROBLEM_REPORT_SYMPTOM is therefore implemented as a junction table between PROBLEM_REPORT and SYMPTOM, supporting a many-to-many relationship. This allows the Diagnosis Engine to operate on structured symptom data rather than relying solely on free-text descriptions.

Report Media

Uploaded images and videos are stored separately in REPORT_MEDIA rather than embedded directly in PROBLEM_REPORT. This avoids overloading the report record and allows multiple media files to be associated with a single report. Each record stores the file type, storage path, and upload timestamp.

Decision Rules

The rule system is divided into DECISION_RULE and RULE_SYMPTOM. DECISION_RULE stores the core rule properties: the associated device, outcome type, guidance text, risk level, priority, whether expert escalation is required, and whether the rule is currently active. RULE_SYMPTOM allows each rule to be associated with multiple symptoms, reflecting the reality that diagnosis typically depends on combinations of observed symptoms rather than a single indicator.

Diagnosis Results

After the engine evaluates a problem report, the result is stored in DECISION_RESULT. This table records which report was evaluated, which rule was matched, the resulting outcome, recommendation text, explanation, confidence score, result date, and final risk level. Storing this record makes the decision process auditable and supports retrospective review of system outputs.

Expert Consultation

When expert assistance is required, a CONSULTATION record is created and linked to the diagnosis result, the requesting user, and the assigned expert. The EXPERT table

includes an AvailabilityStatus attribute that the assignment logic uses to select an available expert automatically. Expert accounts include a PasswordHash field, allowing experts to log in and manage their assigned consultations through the expert dashboard.

The expert dashboard retrieves updates using periodic polling rather than a persistent connection. The frontend queries the consultation status endpoint at a fixed interval to reflect any changes made by the expert.

Replacement Recommendations

Replacement options are stored in REPLACEMENT_OPTION rather than as a single field on DECISION_RESULT, because one diagnosis outcome may correspond to multiple products from different merchants. Each record references the matched rule, the device, and a merchant, and includes product name, type, estimated price, and a direct product URL.

Feedback

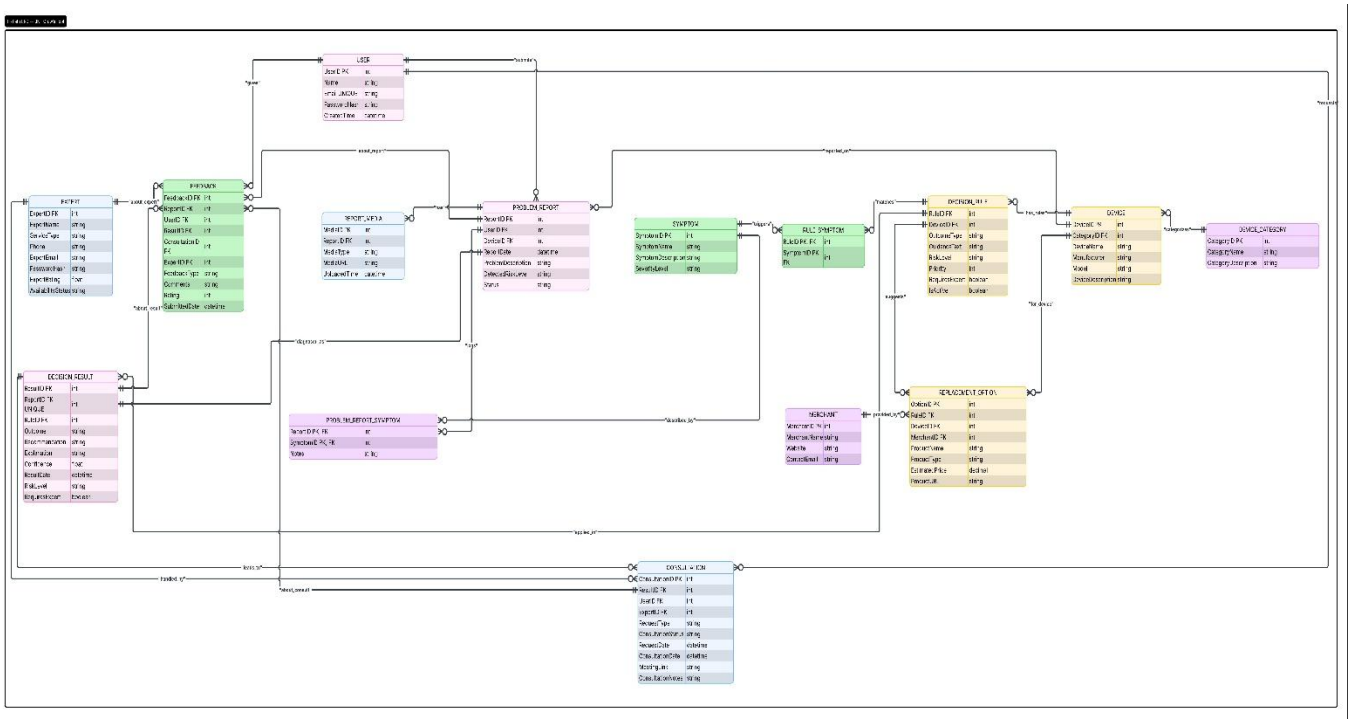
The FEEDBACK table records user ratings and comments after interacting with the platform. The FeedbackType field identifies whether the feedback relates to a diagnosis result or an expert consultation. Consultation feedback triggers an update to the relevant expert's average rating, stored in the EXPERT table.

4.3 Normalization

The schema is designed to satisfy third normal form (3NF) by:

- separating entities with distinct responsibilities into dedicated tables,
- eliminating redundant data storage through foreign key references,
- using junction tables (PROBLEM_REPORT_SYMPTOM, RULE_SYMPTOM) to represent many-to-many relationships,
- storing consultation, media, and replacement data in dedicated tables rather than embedding them in the report or result record.

As shown in the diagram below,



5. Interface Design

The interface design of FixPath focuses on clarity, simplicity, and structured interaction, ensuring that both users and experts can efficiently complete their tasks without unnecessary complexity. The system adopts a workflow-driven design that aligns with the decision-support nature of the platform.

5.1 Design Principles

The interface is designed based on the following principles:

- Clarity: Information is presented in a clear and readable format, avoiding technical jargon wherever possible.
- Consistency: Layout, buttons, and interaction patterns remain consistent across all views.
- Guidance: Users are guided step by step through the troubleshooting process, reducing cognitive load.
- Feedback Visibility: System responses such as diagnosis results, consultation status,

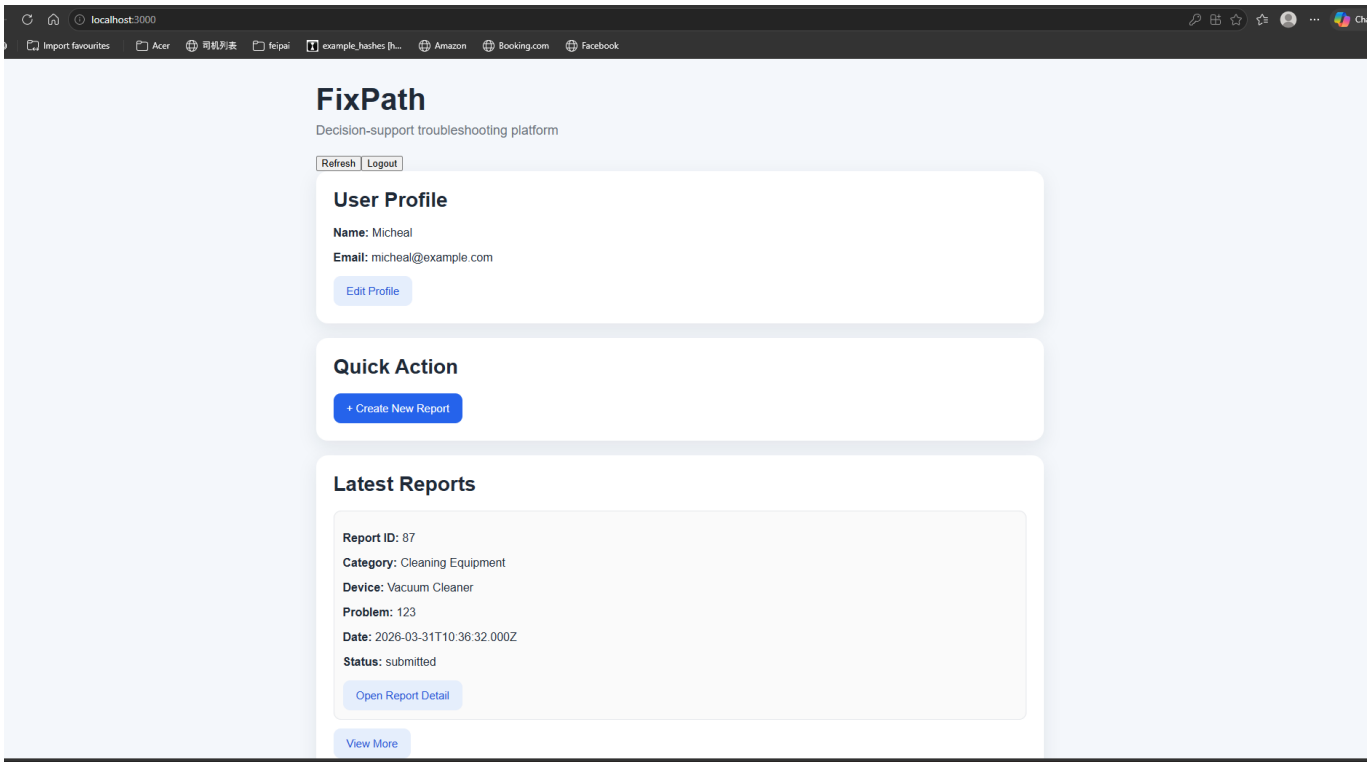
and errors are clearly displayed.

These principles ensure that the interface remains accessible to non-technical users while still supporting complex workflows.

5.2 User Interface (User Side)

The user interface supports the full troubleshooting workflow.

As shown in the diagram below,

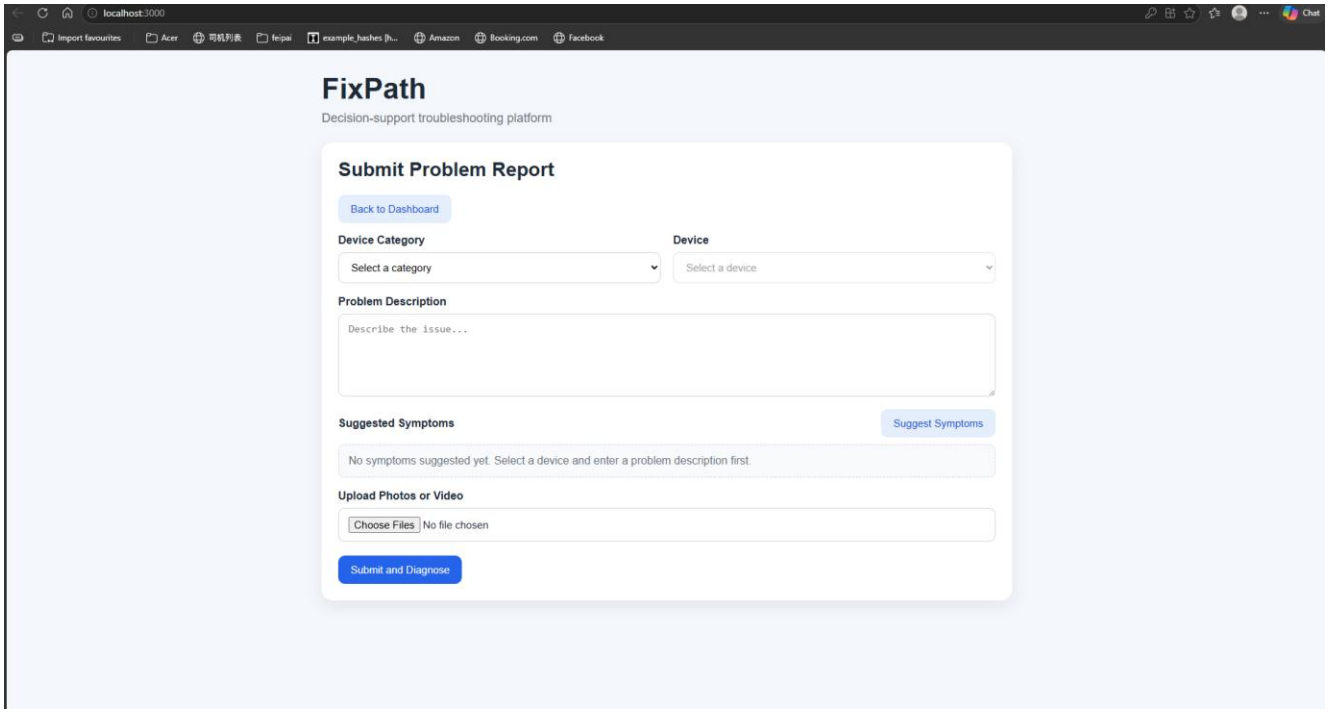


Problem Submission

Users can submit a problem report by selecting a device category, choosing a device, entering a problem description, selecting symptoms, and optionally uploading images or videos.

A keyword-based symptom suggestion feature assists users by recommending relevant symptoms based on the entered description.

As shown in the diagram below,



Diagnosis Result Display

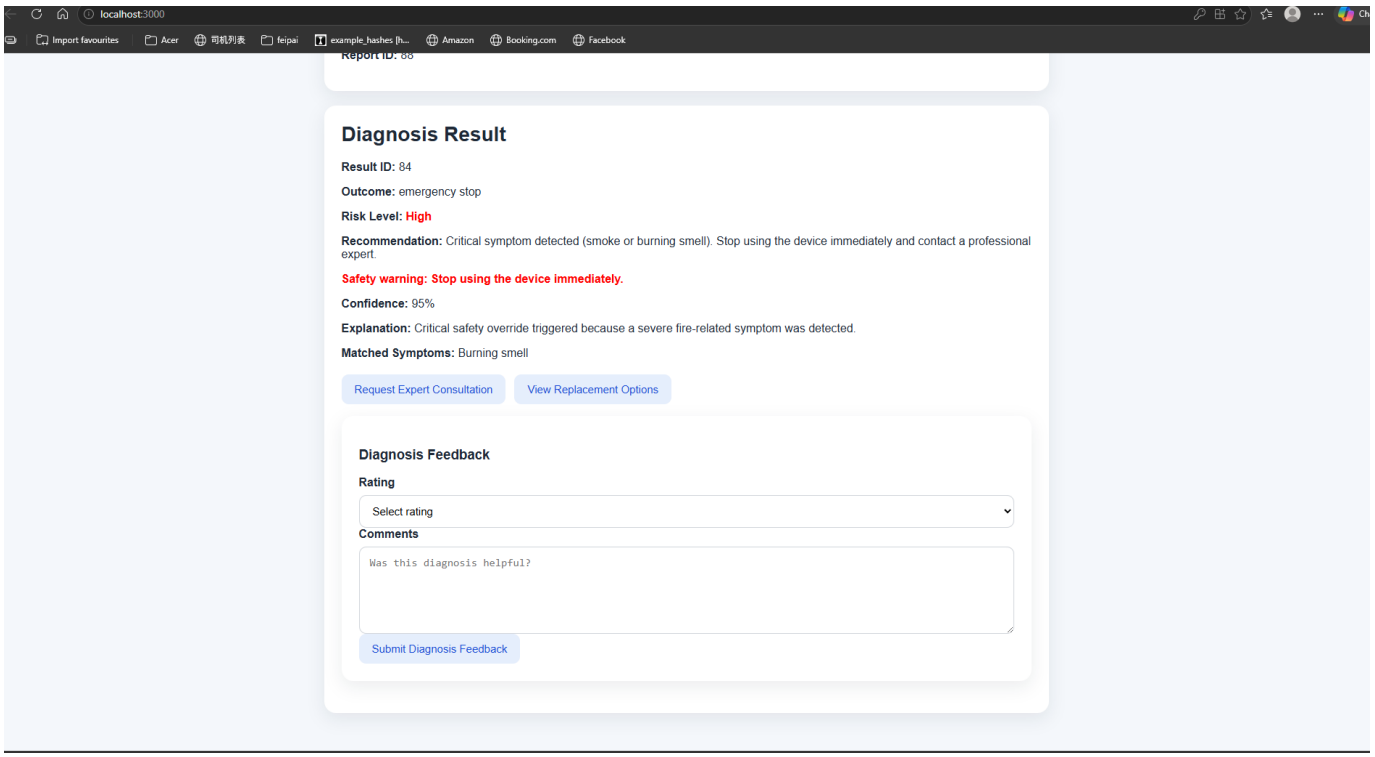
After submission, the system displays:

- risk level
- outcome type
- recommendation
- explanation

Users are then presented with clear next actions, such as:

- follow troubleshooting guidance
- request expert consultation
- view replacement options

As shown in the diagram below,

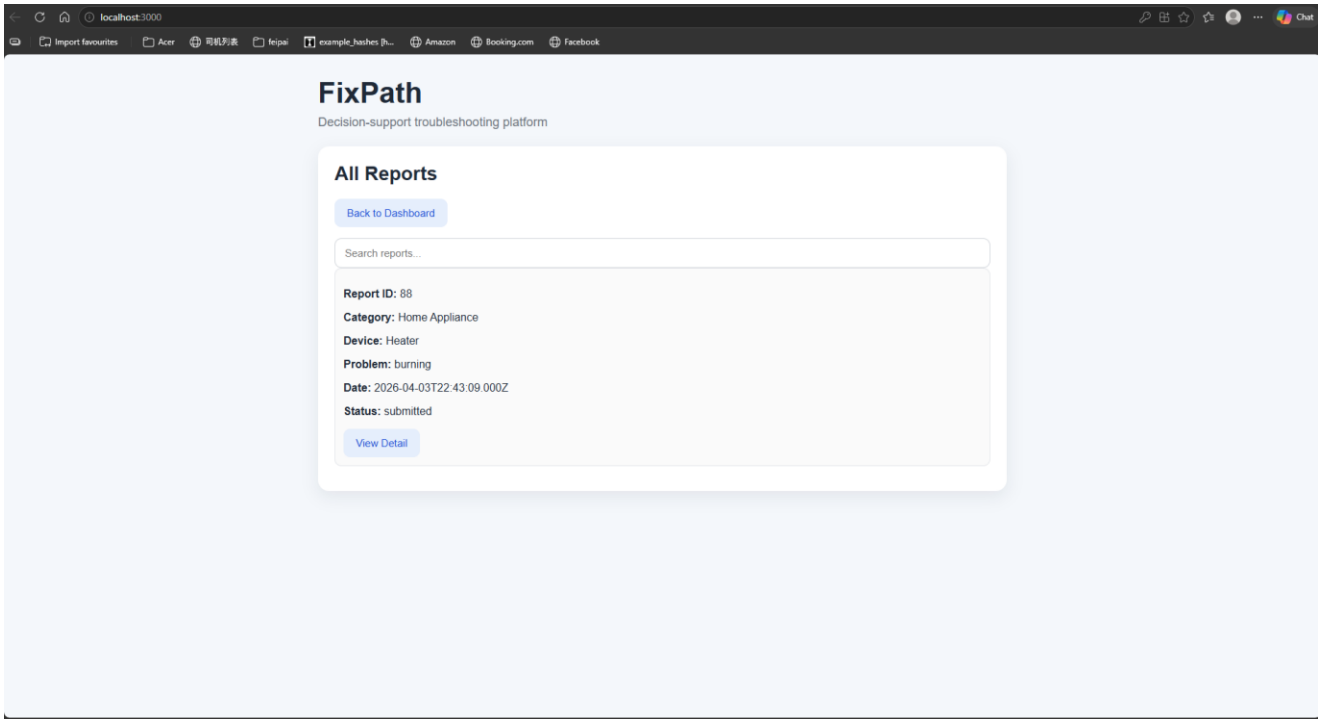


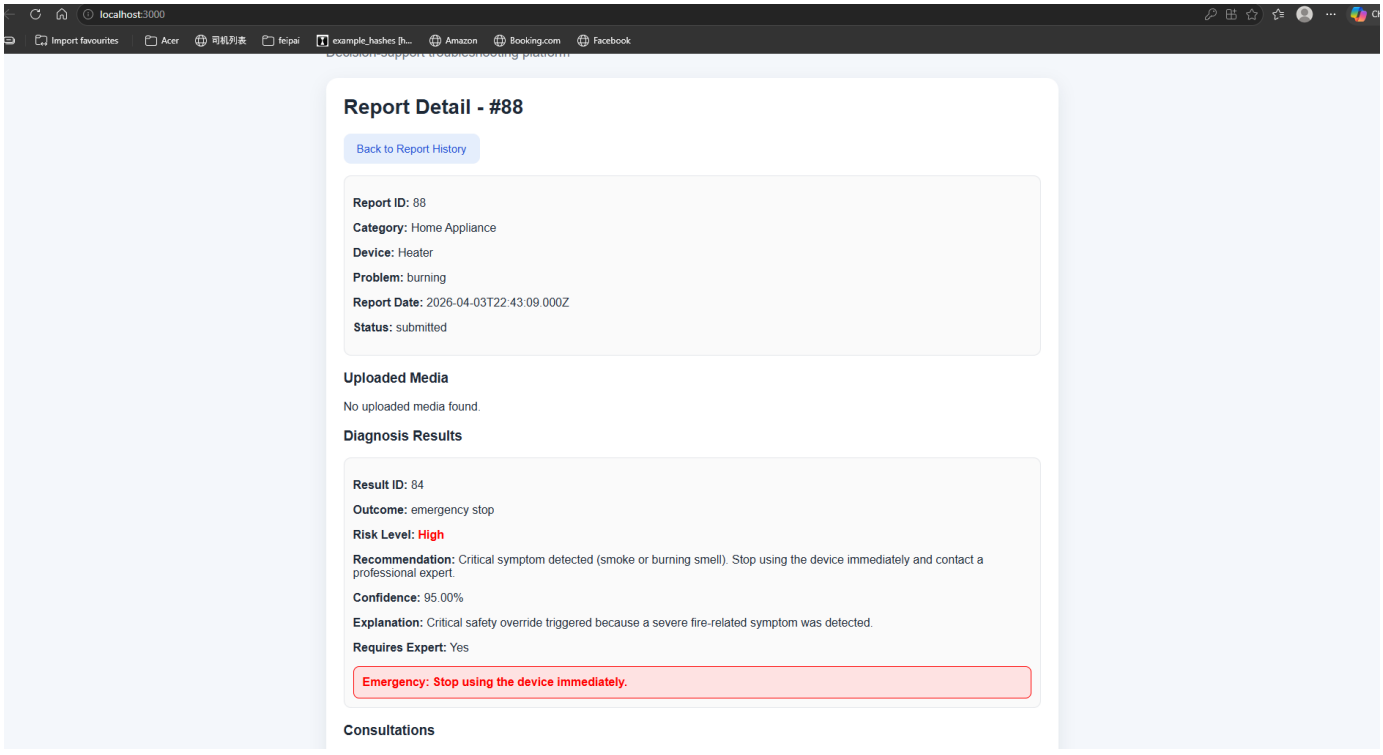
Report History and Detail View

Users can view their previous reports and access detailed information, including uploaded media and diagnosis results.

Media files can be previewed and enlarged for better inspection.

As shown in the diagram below,

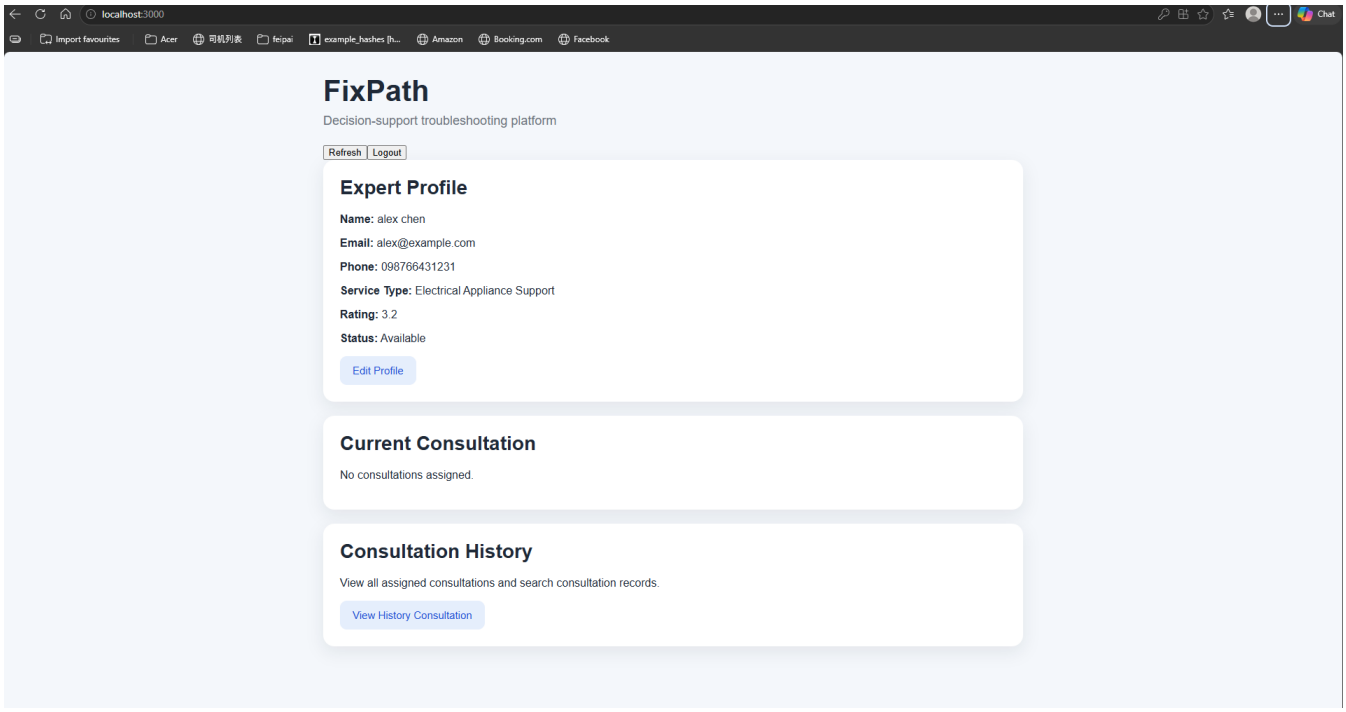




5.3 Expert Interface

The expert dashboard is designed for efficient consultation management.

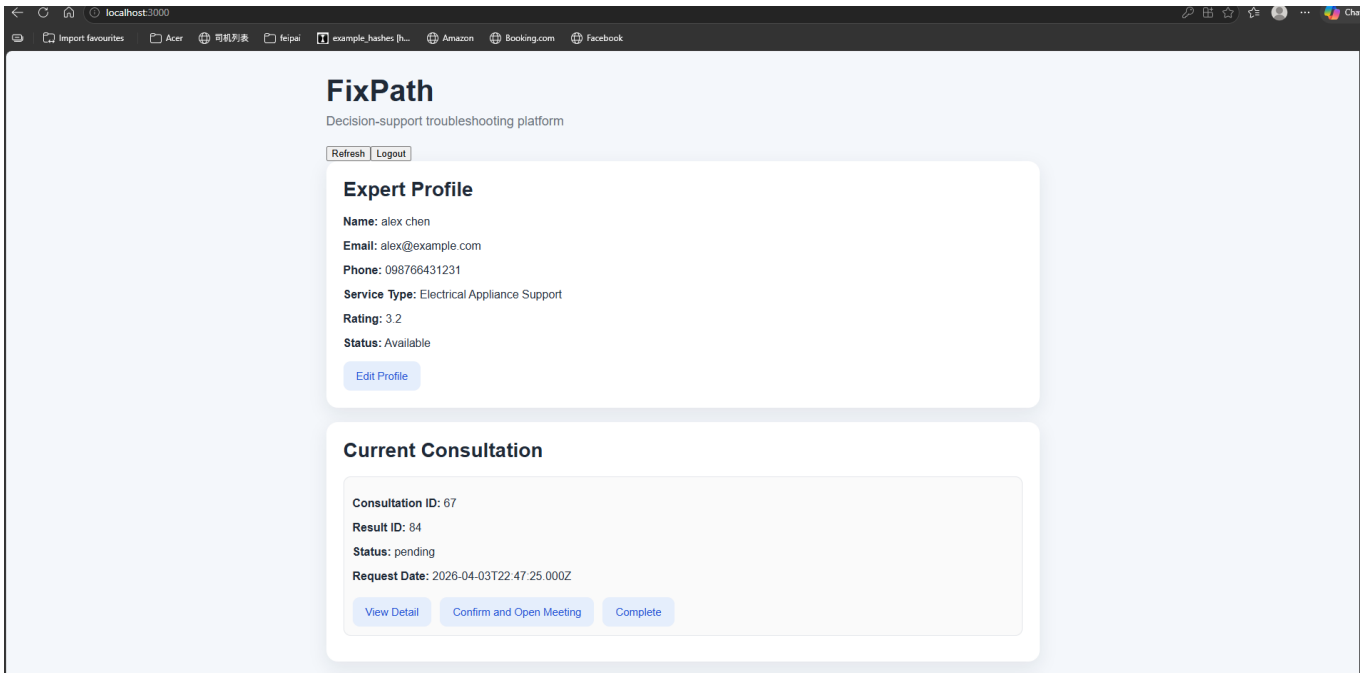
As shown in the diagram below,



Current Consultation

Displays the most recent assigned consultation with key information such as status, request date, and quick action options.

As shown in the diagram below,



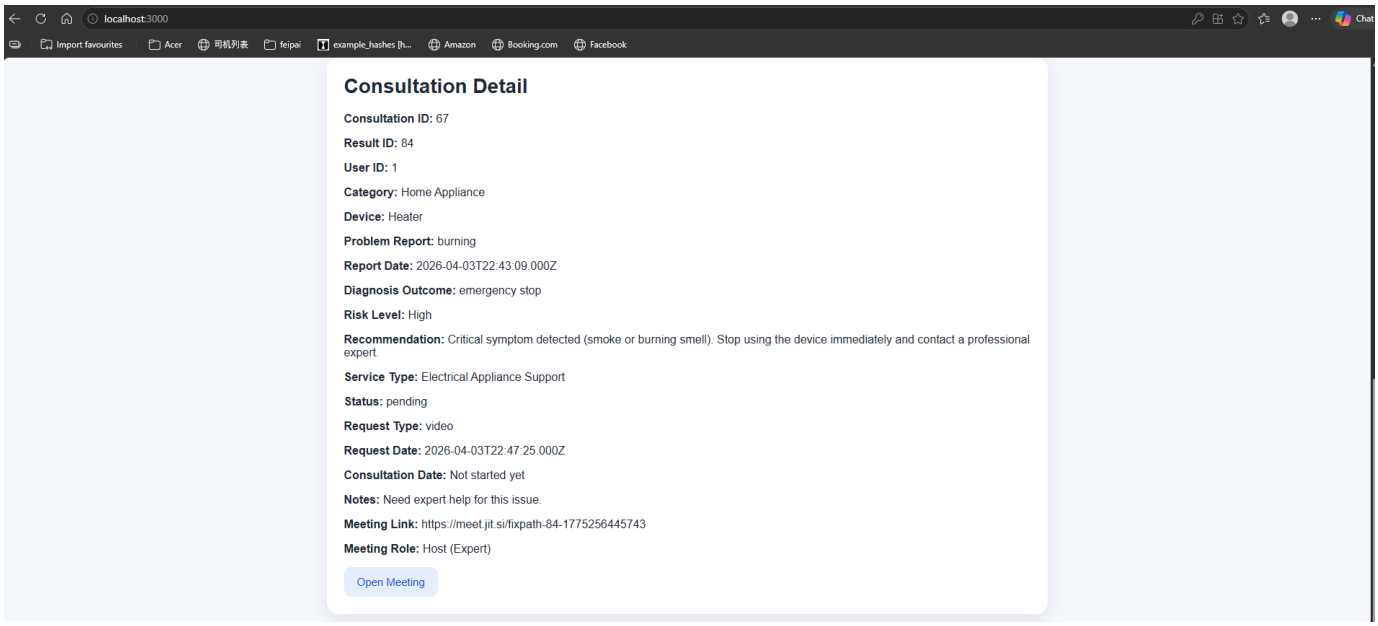
Consultation Detail

Experts can view full details of a consultation, including:

- problem report
- diagnosis result
- risk level and recommendation
- uploaded media (with preview functionality)

This information is positioned directly below the current consultation section to minimize scrolling and improve readability.

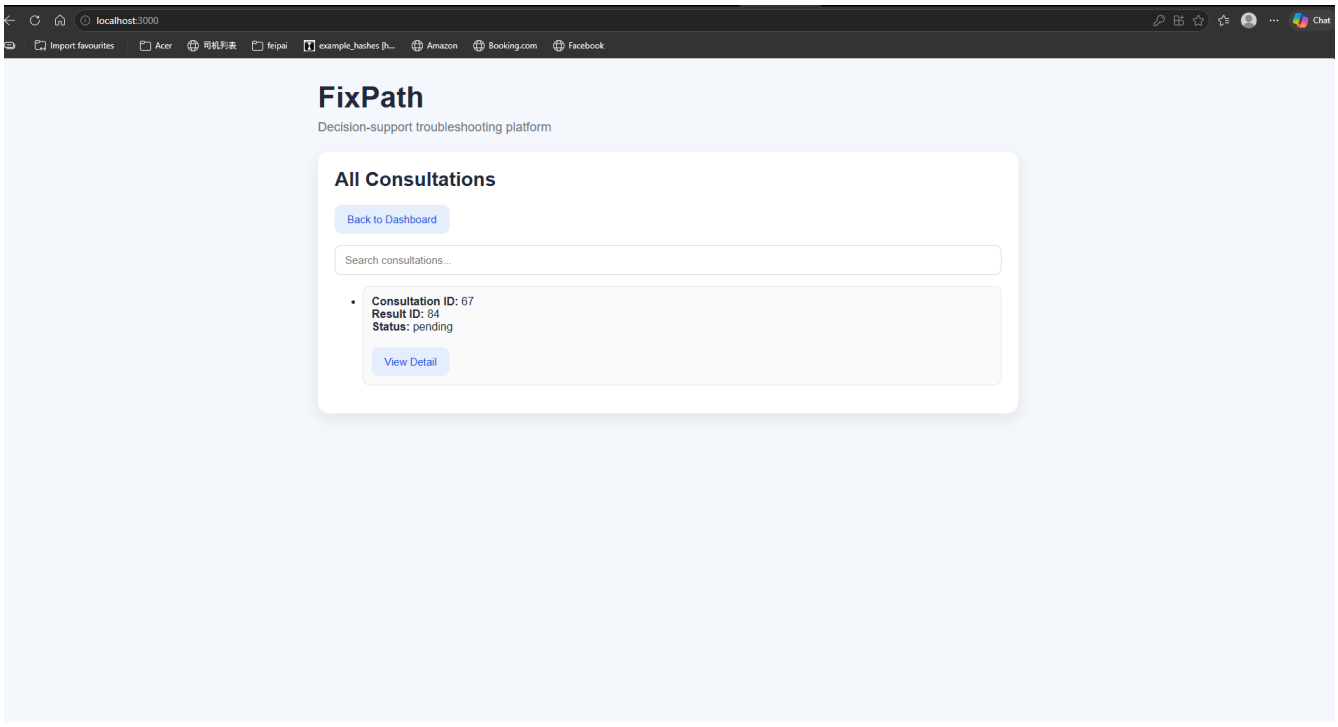
As shown in the diagram below,



Consultation History

Experts can search and review past consultations. A “View Detail” function allows seamless navigation back to the dashboard and automatically scrolls to the detailed view.

As shown in the diagram below,



Real-Time Updates

The system uses periodic polling to refresh consultation data automatically, ensuring that experts receive updated requests without manual refresh.

5.4 Interaction Flow

The interface follows a linear and structured interaction flow:

1. User submits problem report
2. System generates diagnosis result
3. User selects next action (self-repair, expert, or replacement)
4. If expert consultation is requested, the expert receives the request
5. Expert reviews details and updates consultation status

This flow ensures that users and experts operate within a predictable and intuitive system structure.

6. Achieving Non-Functional Requirements

The system design of FixPath satisfies its major non-functional requirements through deliberate architectural and implementation decisions. Rather than only defining these requirements conceptually, the system demonstrates how they are achieved in practice through its workflow, rule-based engine, and modular structure.

6.1 Usability

FixPath achieves usability through a structured, step-by-step interaction model that guides users from problem submission to diagnosis and next-action selection.

The interface reduces cognitive load by avoiding technical terminology and presenting outcomes in clear, plain language. In addition, a keyword-based symptom suggestion module assists users in identifying relevant symptoms based on their problem description, minimizing manual effort and improving accuracy.

This design ensures that even non-technical users can complete the full troubleshooting process without confusion or prior expertise.

6.2 Performance

Performance is achieved using a lightweight rule-based Diagnosis Engine that evaluates only the subset of decision rules associated with the selected device.

The scoring mechanism—based on symptom severity weighting, rule priority, and penalties for mismatched symptoms—enables efficient computation without requiring resource-intensive machine learning models.

By limiting rule evaluation scope and using structured relational queries, the system can return diagnosis results quickly, supporting near real-time interaction suitable for a web-based prototype.

6.3 Scalability

Scalability is supported through the separation of frontend, backend, database, and external integration responsibilities. Each layer communicates through well-defined RESTful APIs, allowing independent scaling and modification.

The rule-based design further enhances scalability, as new devices, decision rules, symptoms, experts, and replacement options can be added directly into the database without requiring changes to backend logic.

This data-driven extensibility ensures that the system can grow in scope and complexity without architectural redesign.

6.4 Security

Security is enforced through multiple layers of protection across the system. Passwords for users and experts are stored as bcrypt-hashed values, preventing exposure of sensitive credentials.

All database operations use parameterized queries, eliminating the risk of SQL injection. Backend input validation ensures that only properly structured data is processed.

Additionally, file uploads are restricted through MIME type validation and file size limits, preventing unsupported or potentially unsafe files from being stored. Database credentials are managed through environment variables and are not exposed in the source code.

Together, these mechanisms ensure secure handling of user data and system operations.

6.5 Safety

Safety is a core requirement of FixPath and is enforced directly within the Diagnosis Engine.

The system includes a safety override mechanism that evaluates high-risk conditions before standard rule scoring. If critical symptoms such as smoke or burning smell are detected, or if multiple high-severity symptoms are present, the system immediately triggers an emergency stop outcome and prevents further self-troubleshooting guidance.

In addition, cases where diagnosis results are ambiguous—defined as situations where top rule scores are too close—are automatically escalated to expert consultation rather than producing uncertain recommendations.

This approach ensures that the system prioritizes user safety and avoids encouraging potentially dangerous actions.

6.6 Maintainability

Maintainability is achieved through a modular system architecture and a normalized relational database design.

The clear separation between frontend, backend, and database layers allows each component to be updated independently without affecting the entire system. Backend logic is organized into functional modules such as diagnosis, consultation management, and symptom suggestion, improving code readability and maintainability.

The database schema follows third normal form (3NF), reducing redundancy and ensuring consistent data relationships. This structure makes it easier to extend or modify system features while preserving stability.

As a result, the system can be efficiently maintained and evolved by future developers.

7. UML Diagrams

UML diagrams are used to provide both structural and behavioral perspectives of the FixPath system. While architectural and database sections describe system components at a high level, UML diagrams illustrate how these components interact and are organized in practice.

The following diagrams collectively present the system from different viewpoints:

7.1 Use Case Diagram

The use case diagram illustrates how different actors interact with the FixPath system and highlights the system's decision-first workflow.

The primary actor is the User, who interacts with the platform to submit problem reports, receive diagnostic results, and choose appropriate next actions. The process begins when the user submits a problem report containing device information, observed symptoms, and a problem description.

After submission, the system performs an automated diagnosis using a rule-based decision engine. Based on the evaluated risk level and rule outcome, the system provides different resolution paths:

- If the issue is classified as low risk, the system provides structured troubleshooting steps that allow the user to attempt self-repair safely.
- If the diagnosis suggests replacement, the user can view recommended replacement options and is redirected to external merchant platforms.
- If the case is high risk or uncertain, the system escalates the request to expert consultation, ensuring that potentially unsafe situations are handled by professionals.

Two additional external actors are involved:

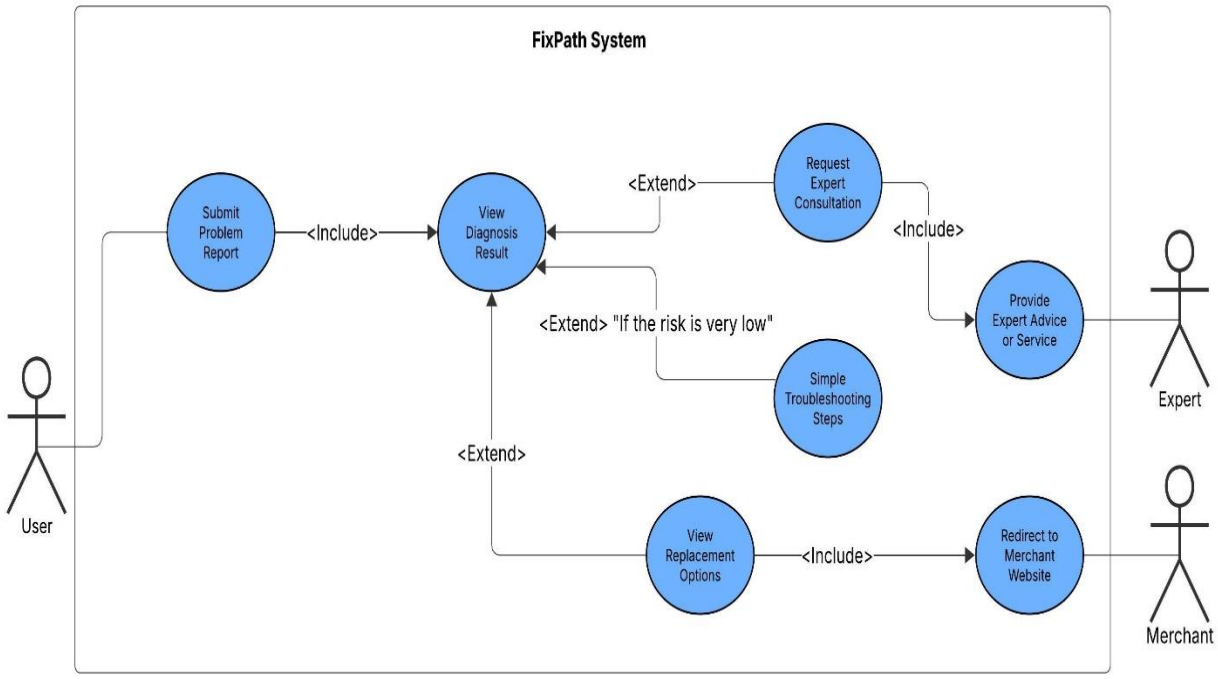
- Expert, who reviews consultation requests, provides professional advice, and updates consultation status.
- Merchant, whose platforms provide product listings when replacement is recommended.

In addition to the main decision flow, the system also supports extended interactions such

as consultation management and user feedback submission, forming a complete end-to-end troubleshooting lifecycle.

Overall, the diagram emphasizes the system's core principle: guiding users from problem reporting to safe and appropriate resolution through structured decision-making rather than direct action.

As shown in the diagram below,



7.2 Class Diagram

The class diagram of the FixPath system represents the structural design of the backend application and illustrates how data entities, business logic, and application interfaces are organized and interact with each other. The system follows a layered architecture consisting of entity classes, service classes, and controller classes, which together support the decision-first troubleshooting workflow.

Entity Classes

Entity classes correspond to the relational database schema and represent the core data

objects used throughout the system. Each entity maps directly to a database table and is responsible for storing persistent information.

The primary entity classes include:

- User and Expert, which represent the two types of system actors. Users submit problem reports, while experts handle consultation requests.

- DeviceCategory and Device, which organize supported devices into structured categories.

- ProblemReport, which captures user-submitted issues, including device information, description, and status.

- ReportMedia, which stores uploaded images or videos associated with a problem report.

- Symptom, which defines observable conditions used as input for diagnosis.

- DecisionRule and RuleSymptom, which together represent the rule-based logic used by the Diagnosis Engine.

- DecisionResult, which records the outcome of the diagnostic process, including recommendation, explanation, and risk level.

- Consultation, which represents expert consultation requests and tracks their lifecycle.

- ReplacementOption and Merchant, which provide product recommendations and external purchase links.

- Feedback, which captures user evaluations of diagnosis results or expert consultations.

These entity classes are connected through foreign key relationships, forming a normalized and structured data model that supports efficient querying and data integrity.

Service Class

The core business logic of the system is encapsulated in the DiagnosisEngine service class.

The DiagnosisEngine is responsible for:

- retrieving relevant decision rules and associated symptoms,
- evaluating rule matches based on symptom input,
- applying safety override conditions for high-risk scenarios,
- calculating rule scores using weighted logic,
- determining the final diagnosis outcome.

By isolating this logic within a dedicated service class, the system ensures that decision-making is consistent, reusable, and independent from the API layer.

Controller Classes

Controller classes handle incoming HTTP requests and act as the interface between the frontend and backend logic. Each controller corresponds to a specific functional module.

The main controllers include:

- reportController, which handles problem report submission and media upload
- diagnoseController, which triggers the diagnosis process
- consultationController, which manages consultation creation and status updates
- replacementController, which retrieves replacement options
- userController and expertController, which handle authentication and account-related operations
- feedbackController, which processes user feedback

Controllers are responsible for input validation, invoking appropriate service functions, and returning structured JSON responses.

Middleware Component

The system also includes a middleware component, UploadMiddleware, which handles file upload processing. This component validates file types and size constraints, ensuring that only supported media formats are accepted.

Class Relationships

The relationships between classes follow a clear layered dependency structure:

- Controller classes depend on service classes to execute business logic.
- Service classes interact with entity classes through database queries.
- Entity classes represent persistent data stored in the database.

Typical interaction flows include:

- reportController creates ProblemReport and ReportMedia records
- diagnoseController invokes DiagnosisEngine to produce a DecisionResult
- consultationController creates Consultation records linked to Expert
- replacementController retrieves ReplacementOption records linked to Merchant

These relationships ensure that responsibilities are clearly separated and that the system remains modular and extensible.

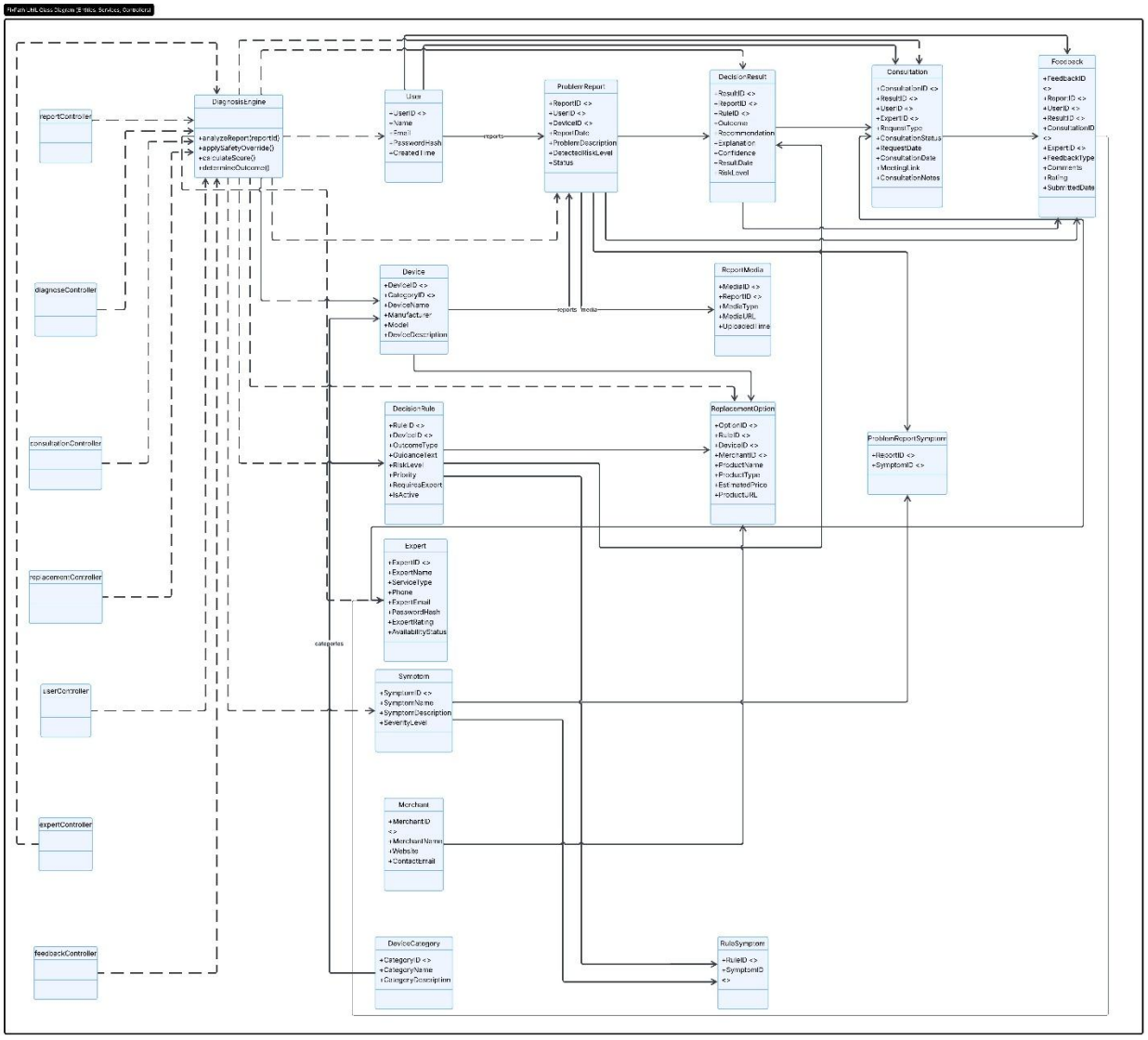
Design Justification

The class structure of FixPath supports key software engineering principles:

- Separation of concerns, by dividing responsibilities across layers
- Maintainability, by isolating business logic from API handling
- Scalability, by allowing new features to be added without restructuring existing components
- Reusability, by centralizing decision logic within the DiagnosisEngine

Overall, the class diagram reflects a well-structured backend architecture that aligns with the system's decision-first design philosophy and supports its core functionality effectively.

As shown in the diagram below,



The class diagram complements the sequence diagrams by presenting the static structure of the system, while sequence diagrams illustrate dynamic interactions between components.

7.3 Sequence Diagrams

The sequence diagrams illustrate the dynamic interactions between system components during key operational scenarios in the FixPath platform.

These diagrams demonstrate how user actions trigger backend processing, how the rule-based Diagnosis Engine evaluates reports, and how the system returns structured results

and next actions.

Three sequence diagrams are presented to represent the main system workflows:

1. The main troubleshooting and diagnosis process
2. The expert consultation request and handling process
3. The replacement recommendation and redirection process

Together, these diagrams show how FixPath orchestrates user interaction, backend processing, and external services to support a complete decision-support workflow.

Sequence Diagram 1: Main Diagnosis Process

This sequence diagram illustrates the core workflow of the FixPath system, from problem submission to diagnosis result generation.

The sequence begins with a user-initiated request, which triggers backend processing through the report submission API. The report includes device information, selected symptoms, and a textual problem description.

The frontend sends a request to the backend system, where a new ProblemReport record is created and stored in the database.

The backend then invokes the Diagnosis Engine to analyze the report. Before standard rule evaluation begins, the system performs a safety override check to detect critical conditions, such as high-risk symptoms or combinations of severe symptoms. If such conditions are detected, the system immediately assigns an emergency outcome and bypasses further rule evaluation.

If no safety override is triggered, the engine retrieves all relevant DecisionRule records associated with the reported device and loads their corresponding symptoms. The engine then compares the reported symptoms with rule conditions and calculates scores using weighted severity, rule priority, and matching accuracy.

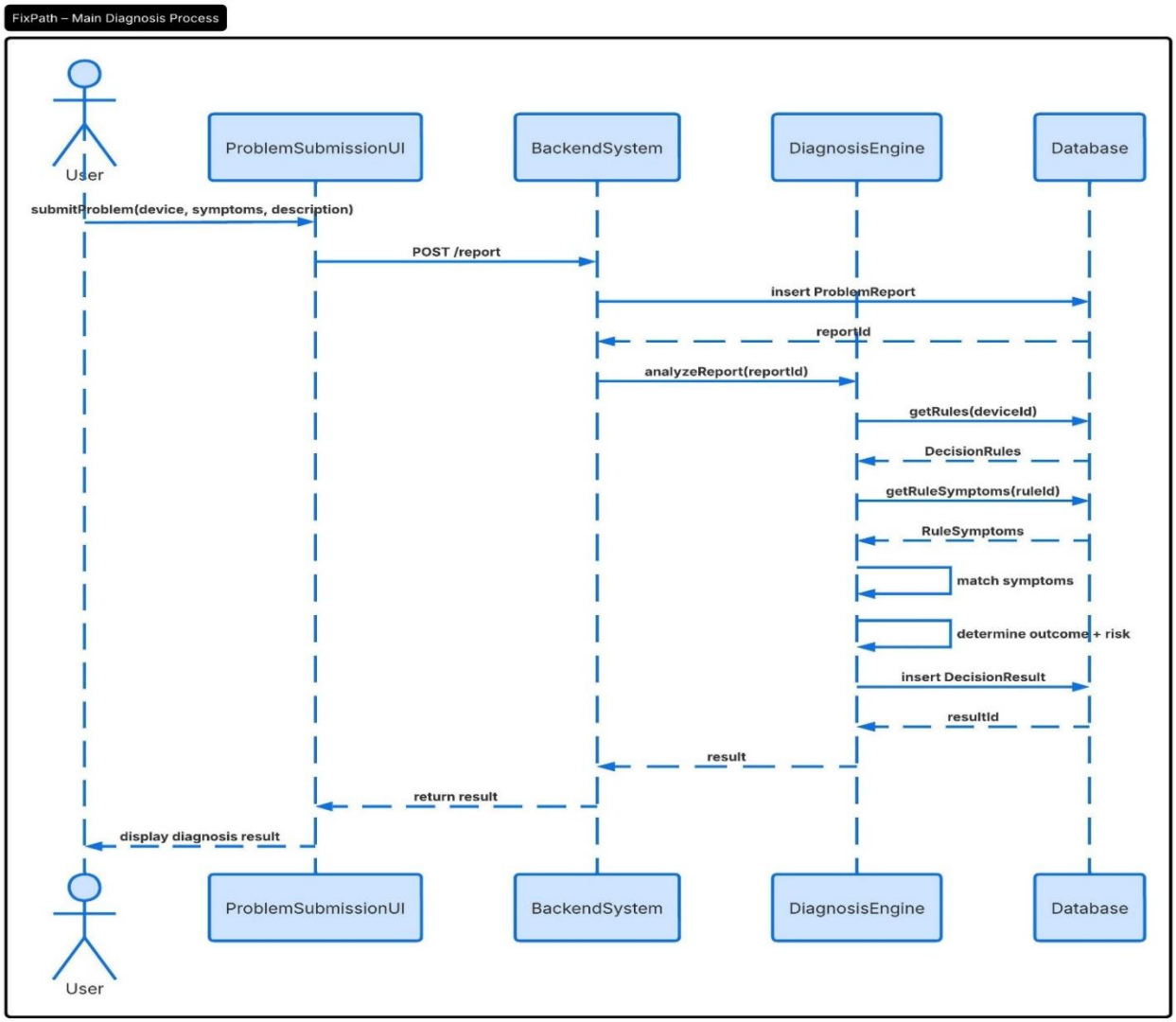
Based on this evaluation, the system determines the most appropriate outcome and assigns a corresponding risk level. A DecisionResult record is generated and stored in the database.

Finally, the backend returns the diagnosis result to the frontend, where it is displayed to

the user along with explanations and recommended next actions.

This sequence demonstrates how the system transforms user input into structured, safety-aware diagnostic output through rule-based evaluation.

As shown in the diagram below,



Sequence Diagram 2: Expert Consultation Process

This sequence diagram illustrates how the system handles cases that require professional assistance.

The process begins when the user selects the option to request expert consultation after receiving a diagnosis result. The frontend sends a request to the backend, which creates a new Consultation record linked to the corresponding diagnosis result and user.

The backend then queries the database to identify an available expert based on availability status. Once an expert is selected, the consultation is assigned and stored.

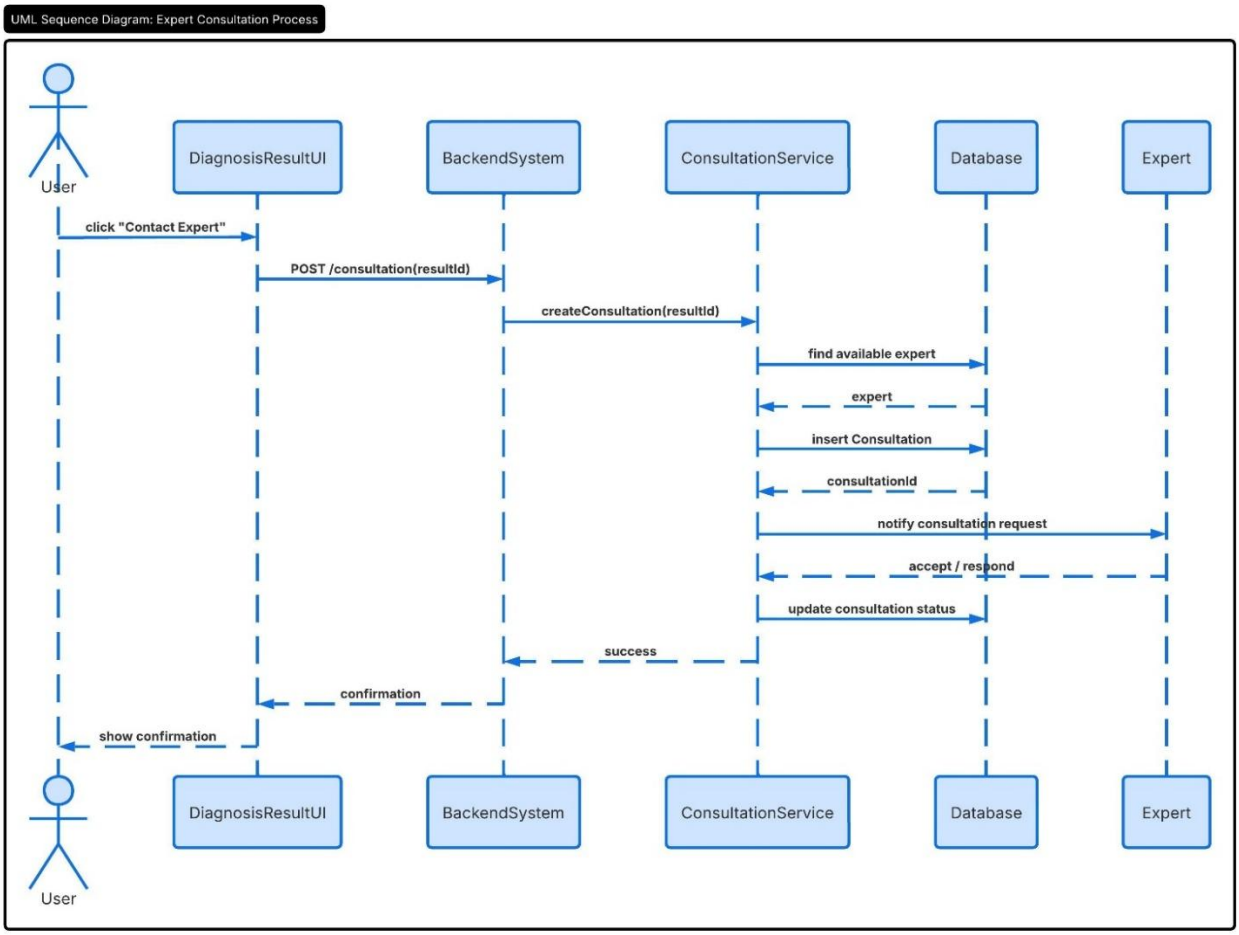
Instead of using real-time push notifications, the expert interface retrieves consultation requests through periodic polling, where the frontend automatically fetches updated consultation data at regular intervals. This ensures that newly assigned consultations are visible without requiring manual refresh.

The expert reviews the consultation details, including the original problem report, diagnosis result, and uploaded media. The expert can then accept the request and update the consultation status (e.g., ongoing or completed).

The backend records status updates and returns confirmation to the frontend, which displays updated information to both the expert and the user.

This sequence highlights how FixPath manages expert involvement while maintaining a simple and scalable communication model.

As shown in the diagram below,



Sequence Diagram 3: Replacement Recommendation Process

This sequence diagram illustrates how the system provides replacement options when the diagnosis result suggests that repair is not appropriate.

The process begins when the user selects the option to view replacement recommendations. The frontend sends a request to the backend using the diagnosis result identifier.

The backend invokes the Replacement Service, which queries the database for ReplacementOption records associated with the matched decision rule and device. The system also retrieves corresponding merchant information, linking each product to a specific vendor.

The backend returns a structured list of replacement options to the frontend, where they

are displayed as product cards, including product names, estimated prices, and external links.

When the user selects a product, the system redirects the user to the merchant's website, where the purchase process is completed outside of the FixPath platform.

This sequence demonstrates how the system integrates external services while maintaining a clear boundary between decision support and transaction execution.

As shown in the diagram below,

